Extra topic I: pickling

`pickle` in Python helps you save a copy of data structures you work with in your current environment.

Saving a single data structure

Simple data structures like `str`, `int`, `float`, or functions can be saved using `pickle.dump`.

In [1]:
```
import pandas as pd

# A variable: list
topics = [
    "Getting Started with Python",
    "Tabular Data with Pandas I",
    "Tabular Data with Pandas II",
    "Plotting with Matplotlib",
    "Control Flow & Functions",
    "Error Handling & Debugging"
]

# A variable: string
name = 'Jin'

# A variable: float
py_version = 3.12

# A function
def your_favorite_topic(topics):
    """Return your favorite topic from the list."""
    return topics[4]

# Function call
your_favorite_topic(topics)
```

Out[1]:
```
'Control Flow & Functions'
```

In [2]:
```
# Import the module first
# 'pkl' is an alias for 'pickle'
import pickle as pkl

# `pickle.dump()` can save a data structure to a file.
# You provide a data structure as `obj`
# and a file object as `file`.
# A file object is created using `open()`.
```

```
pkl.dump(obj=name,
         file=open('name.pkl', 'wb'))
```

Running the line above should create `name.pkl` in your current working environment. Check the folder listed here:

In [3]:
```
# Save the list object
pkl.dump(obj=topics,
         file=open('topics.pkl', 'wb'))

# This is more typical usage:

# `with` automatically closes the file after writing
# We are opening the file in 'wb' mode: write binary
# and saving the open file object as `f`
with open('your_favorite_topic.pkl', 'wb') as f:
    # Again, `obj` and `file` arguments are provided
    # For a function, just give the function name
    # without parentheses
    pkl.dump(your_favorite_topic,f)
```

In [4]:
```
# This is how you would load a picked data structure
# back into memory.
# Similarly, use `with` to open the file in 'rb' mode:
# read binary and save the file object as `func`.
# People often use 'f' to mean 'file',
# but you can use any valid variable name
# as we're doing here with 'func'.
with open('your_favorite_topic.pkl', 'rb') as func:
    # Use `pkl.load()` to load the data structure
    # from the file object `func`.
    # Also, you can name the function differently.
    # Here, we name it `get_favorite_topic`.
    get_favorite_topic = pkl.load(func)

with open('topics.pkl', 'rb') as tp:
    loaded_topics = pkl.load(tp)

# Is the `loaded_topics` same as `topics`?
print(loaded_topics)
```

```
['Getting Started with Python', 'Tabular Data with Pandas I', 'Tabular
Data with Pandas II', 'Plotting with Matplotlib', 'Control Flow & Funct
ions', 'Error Handling & Debugging']
```

In [5]:
```
# Is the loaded function (get_favorite_topic)
# same as your_favorite_topic?
# We can tell by the work of it.
# If it's the same, it should return the same result
```

```
# as your_favorite_topic(topics)
# because `loaded_topics` is the same as `topics`.
get_favorite_topic(loaded_topics)
```

Out[5]: 'Control Flow & Functions'

In [6]:
```
# A DataFrame / Series can be saved as well.
df = pd.DataFrame(topics, columns=["Topics"])
# Use `to_pickle()` method of DataFrame/Series
df.to_pickle("topics_df.pkl")

# Load it back: use `pd.read_pickle()`
loaded_df = pd.read_pickle("topics_df.pkl")
loaded_df
```

Out[6]:

	Topics
0	Getting Started with Python
1	Tabular Data with Pandas I
2	Tabular Data with Pandas II
3	Plotting with Matplotlib
4	Control Flow & Functions
5	Error Handling & Debugging

Saving your entire workspace?

Your **entire** workspace can be accessed using `globals()`

In [7]:
```
# `globals()` returns a dictionary of all variables
# in the current global scope
# that you're not too familiar with.
# I will revisit, but you MUST make a COPY.
all_variables = globals().copy()

# Example: access the variable `name`
all_variables['name']
```

Out[7]: 'Jin'

OK. Then can we use `.dump()` on `all_variables`?

Not quite, because not all variables of `globals()` can be pickled. You may not know which can be pickled or not. This is where `try/except` can be useful!

In [8]:
```
# For example, '__builtin__' of `all_variables`
```

```
# is not picklable.
pkl.dump(all_variables, open('all_variables.pkl', 'wb'))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[8], line 3
      1 # For example, '__builtin__' of `all_variables`
      2 # is not picklable.
----> 3 pkl.dump(all_variables, open(                    ,     ))

TypeError: cannot pickle 'module' object
```

In [9]:
```
# We can check which variables can be pickled
# by using `.dumps()` in a `try/except` block.

# First, see what `.dumps()` does.
# It returns the pickled byte stream of the object.
# `name` is a string object (value = 'Jin')
pkl.dumps(all_variables['name'])
```

Out[9]:
```
b'\x80\x04\x95\x07\x00\x00\x00\x00\x00\x00\x00\x8c\x03Jin\x94.'
```

In [10]:
```
# So if `pkl.dumps(val)` works for a variable,
# it means the variable is picklable.
# In contrast, if it returns an error,
# the variable is not picklable.
pkl.dumps(all_variables['__builtin__'])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[10], line 5
      1 # So if `pkl.dumps(val)` works for a variable,
      2 # it means the variable is picklable.
      3 # In contrast, if it returns an error,
      4 # the variable is not picklable.
----> 5 pkl.dumps(all_variables[             ])

TypeError: cannot pickle 'module' object
```

So we can iterate through `all_variables` using its keys, and save key-value pairs that are *picklable* to a new dictionary (ex. `to_save`). Once the new dictionary is prepared, we can use `.dump()` to pickle all picklable.

In [11]:
```
# This is a dictionary to store picklable variables:
to_save = {}

# `all_variables` is a dictionary
for name, val in all_variables.items():
```

```
    try:
        pkl.dumps(val)  # Try to pickle the variable
        to_save[name] = val  # If successful, add to to_save
    # Not sure what exceptions might occur,
    # so catch all exceptions
    except Exception as e:
        print(f"Could not pickle {name}: {e}")
```

```
Could not pickle __builtin__: cannot pickle 'module' object
Could not pickle __builtins__: cannot pickle 'module' object
Could not pickle get_ipython: Can't pickle local object 'ScriptMagics._
make_script_magic.<locals>.named_script_magic'
Could not pickle exit: Can't pickle local object 'ScriptMagics._make_sc
ript_magic.<locals>.named_script_magic'
Could not pickle quit: Can't pickle local object 'ScriptMagics._make_sc
ript_magic.<locals>.named_script_magic'
Could not pickle open: Can't pickle <function open at 0x10681c220>: it'
s not the same object as _io.open
Could not pickle pd: cannot pickle 'module' object
Could not pickle pkl: cannot pickle 'module' object
Could not pickle f: cannot pickle 'BufferedWriter' instances
Could not pickle func: cannot pickle 'BufferedReader' instances
Could not pickle tp: cannot pickle 'BufferedReader' instances
```

In [12]:
```
# Successfully pickled
with open('picklable_variables.pkl', 'wb') as f:
    pkl.dump(to_save, f)
```

Previously we prepared `all_variables` by using `.copy()` method.

```
>>> all_variables = globals().copy()
```

It may be a minor thing, but if you don't do it, the dictionary we created to save **picklable** items would also be saved in the pickled item.

In [13]:
```
# Load 'picklable_variables.pkl'
with open('picklable_variables.pkl', 'rb') as f:
    pickle_without_new_dict = pkl.load(f)

# Was `to_save` in the pickled item?
# The answer is No (False)
'to_save' in pickle_without_new_dict
```

Out[13]:
```
False
```

In [14]:
```
# Let's start over - but this time without copying
again_all_variables = globals()

# First check if `again_all_variables` has a key:
# 'to_save_new'
# which is the name of the new dictionary
```

```
# we will make below.
print('to_save_new' in again_all_variables)
```

```
False
```

In [ ]:
```
# Make `to_save_new` dictionary
# and then check again
to_save_new = {}

# Oh, 'to_save_new' is now in globals()
print('to_save_new' in again_all_variables)
```

```
True
```

In [16]:
```
for name, val in again_all_variables.items():
    try:
        pkl.dumps(val)
        to_save_new[name] = val
    except Exception as e:
        print(f"Could not pickle {name}: {e}")

# Pickle the new dictionary
with open('new_picklable_variables.pkl', 'wb') as f:
    pkl.dump(to_save_new, f)

# Load and check if `to_save_new` was pickled as well
with open('new_picklable_variables.pkl', 'rb') as f:
    pickle_with_new_dict = pkl.load(f)

# `to_save_new` was saved, so the answer
# should be Yes (True)
'to_save_new' in pickle_with_new_dict
```

```
Could not pickle __builtin__: cannot pickle 'module' object
Could not pickle __builtins__: cannot pickle 'module' object
Could not pickle get_ipython: Can't pickle local object 'ScriptMagics._
make_script_magic.<locals>.named_script_magic'
Could not pickle exit: Can't pickle local object 'ScriptMagics._make_sc
ript_magic.<locals>.named_script_magic'
Could not pickle quit: Can't pickle local object 'ScriptMagics._make_sc
ript_magic.<locals>.named_script_magic'
Could not pickle open: Can't pickle <function open at 0x10681c220>: it'
s not the same object as _io.open
Could not pickle pd: cannot pickle 'module' object
Could not pickle pkl: cannot pickle 'module' object
Could not pickle f: cannot pickle 'BufferedReader' instances
Could not pickle func: cannot pickle 'BufferedReader' instances
Could not pickle tp: cannot pickle 'BufferedReader' instances
Could not pickle all_variables: cannot pickle 'module' object
Could not pickle again_all_variables: cannot pickle 'module' object
```

Out[16]: True

Consider making a function to modularize

In [ ]:
```python
def save_workspace(filename: str = 'all_variables.pkl'):
    """
    Save all picklable workspace variables
    up to the point this function is called.

    Parameters
    ----------
    filename: str
        Name of the pickle file. Default to 'all_variables.pkl'

    Returns
    -------
    None
    """
    # Make sure that the extension is ".pkl"
    # Can you recognize how `assert` is used in this case?
    assert isinstance(filename, str) and filename[-4:] == '.pkl', \
    "Make sure you are providing a string with '.pkl' extension."

    all_variables = globals().copy()

    to_save = dict()
    for name, val in all_variables.items():
        try:
            pkl.dumps(val)
            to_save[name] = val
        except Exception as e:
            # Just pass
            pass

    with open(filename, 'wb') as f:
        pkl.dump(to_save, f)
```

In [19]:
```python
# This is how you will run it:
# >>> save_workspace('my_pickle.pkl')

# This will return an error:
save_workspace('my_pickle.txt')
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[19], line 5
      1 # This is how you will run it:
      2 # >>> save_workspace('my_pickle.pkl')
      3
      4 # This will return an error:
----> 5 save_workspace(                )

Cell In[18], line 16, in save_workspace(filename)
      2 """
      3 Save all picklable workspace variables
      4 up to the point this function is called.
   (...)     13 None
     14 """
     15 # Make sure that the extension is ".pkl"
---> 16 assert isinstance(filename, str) and filename[-4:] == '.pkl', \
     17 "Make sure you are providing a string with '.pkl' extension."
     19 all_variables = globals().copy()
     21 to_save = dict()

AssertionError: Make sure you are providing a string with '.pkl' extension.
```

Summary

Pickling is a convenient way to save your entire Python workspace. However, when collaborating with others, storing everything in a single `.pkl` file is often **not** ideal.

- Your collaborators must have a compatible Python environment to open the `.pkl` file.
- When unpickling, Python attempts to import the **exact modules and versions** that were used when the file was created.
  For instance, this notebook was created with `numpy==2.3.3`. If someone tries to load it in an environment running a different version (e.g., `numpy==2.1.0`), the file will fail to load because the internal structure of NumPy has changed.